

ACOL 226

April 8th

plus :: $\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})$

$(\text{plus } m)n = m + n$

plus-m Int $\rightarrow$ Int

plus-m n = n + m

plus : $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$
 $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$

$$\text{plus3} :: \underline{\text{Int}} \rightarrow \underline{(\underline{\text{Int}} \rightarrow (\underline{\text{Int}} \rightarrow \underline{\text{Int}}))}$$

$$((\text{plus3 } k) l) m = k + l + m$$

$$\text{plus3} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$$

$$\text{plus3 } k \ l \ m = k + l + m$$

$\text{Xor} :: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$

$$\text{Xor } b1 \ b2 = (b1 \ \underline{\text{and}} \ (\text{not } b2)) \ \underline{\text{or}} \ (\text{not } b1)$$

$$(b2 \ \text{and} \ (\text{not } b1))$$

pattern
- matching

$\text{Xor} :: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$

→	Xor	<u>True</u>	<u>False</u>	$=$	True	} or
→	Xor	<u>False</u>	<u>True</u>	$=$	True	
→	Xor	<u>b1</u>	<u>b2</u>	$=$	<u>False</u>	

Xor False False

Inductive Definitions

f

$\left\{ \begin{array}{l} f(0) \\ f(n) \end{array} \right.$

explicitly

in terms of $f(0), f(1), \dots, f(n-1)$

$$\left\{ \begin{array}{l} \boxed{\text{fact } 0 = 1} \\ \text{fact } n = n * (\text{factorial } (n-1)) \end{array} \right.$$

these brackets are necessary because function applications bind more tightly than arithmetic operators.

Conditional Definitions

fact :: Int → Int

fact 0 = 1

fact n | $\frac{n < 0}{n > 0}$

guards

→

→

can overlap

need not be
exhaustive

fact (-n)

n * fact (n-1)

fact :: Int $\rightarrow$ Int

fact 0 = 1

fact n | n < 0 = fact (-n)

| n > 1 = n * fact (n-1)



these guards are not
exhaustive
(fact 1 cannot be computed)

fact 1 = 1

One can also use "otherwise" to exhaust the possibilities in a conditional defⁿ, like this

`xor :: Bool → Bool → Bool`

<code>xor b1 b2</code>	<code> </code>	<code>(b1 && (not b2)) == True</code>
	<code> </code>	<code>(b2 && (not b1)) == True</code>
	<code> </code>	<code>otherwise == False</code>

How does Haskell "compute"?

$$(a+b)^2 = a^2 + 2ab + b^2$$

$(a+b)^2$ $\xrightarrow{\text{rewritten}}$

by doing rewriting of this form

fact 3

$$\leadsto 3 * \text{fact } (3-1)$$

$$\leadsto 3 * \text{fact } (\underline{\underline{2}})$$

$$\leadsto 3 * (2 * \text{fact } (2-1))$$

$$\leadsto 3 * (2 * (\text{fact } 1))$$

$$\leadsto 3 * (2 * (1 * \text{fact } (1-1)))$$

$$\leadsto 3 * (2 * (1 * (\underline{\underline{\text{fact } 0}})))$$

$$\leadsto 3 * (2 * (1 * (1)))$$

$$\leadsto 6$$

power :: Float $\rightarrow$ Int $\rightarrow$ Float

power x 0 = 1

power x n | $n > 0$ = $x * (\text{power } x (n-1))$

power (4.0/0.0) 0

$\rightarrow$ evaluates to 1

Haskell uses "lazy" simplification $\rightarrow$

is actually needed to evaluate the function. it does not simplify the argument to a function until the value of the argument

Lists

[Int] the
type of list
where the
elements are of
type Int.

→ [1, 2, 3, 4, 5]

[True, False, True, True, False]

[[1, 2], [3, 4, 5], [6, 7]]

[[1, 2, True], [False, 4]]

[[1, 3], (7)]

[1, 2, 3]*

[1 : (2 : (3 : []))]

list building operator

[1, 2, 3]

1 : [2, 3]

1 : 2 : [3]

1 : 2 : 3 : []

all same lists

$\text{length} :: [\text{Int}] \rightarrow \text{Int}$

$\text{length } [] = 0$

$\rightarrow \text{length } l = 1 + \underline{\underline{(\text{length } (\text{tail } l))}}$



$\text{sum} :: [\text{Int}] \rightarrow \text{Int}$

$\text{sum} [] = 0$

$\text{sum } l = (\text{head } l) + (\text{sum } (\text{tail } l))$

$\text{sum } (x:xs) = x + (\text{sum } xs)$

$\left\{ \begin{array}{l} \text{length} :: [\text{Int}] \rightarrow \text{Int} \end{array} \right.$

$\text{length} [] = 0$

$\text{length } (x:xs) = 1 + (\text{length } xs)$

append :: [Int] → [Int] → [Int]

append to [3, 2] the list [4, 5, 6]
to get
[3, 2, 4, 5, 6]

append [] ys = ys

append (x:xs) ys = x : (append xs ys)

[1, 2, 3] ++ [4, 5] = [1, 2, 3, 4, 5]
built-in operator in Haskell

take n l

returns a list
containing the first n
elements of l

drop n l

returns the list
obtained by removing
the first n element
from l

For any list l and any integer n , we
are guaranteed that
$$l = (\text{take } n \text{ } l) ++ (\text{drop } n \text{ } l).$$